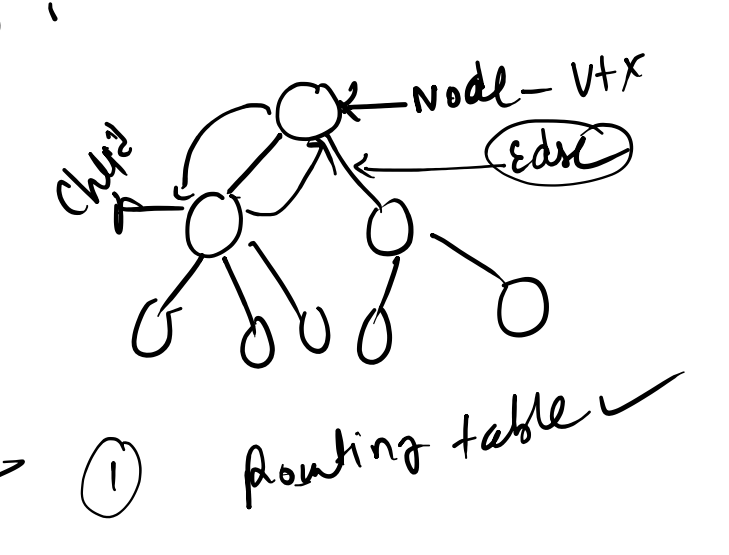
Lec-32
13/09/25 9:15 AM

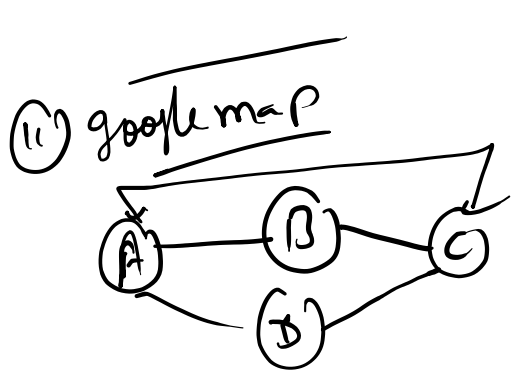
Graph

defn: it is finite set Edge Vtx

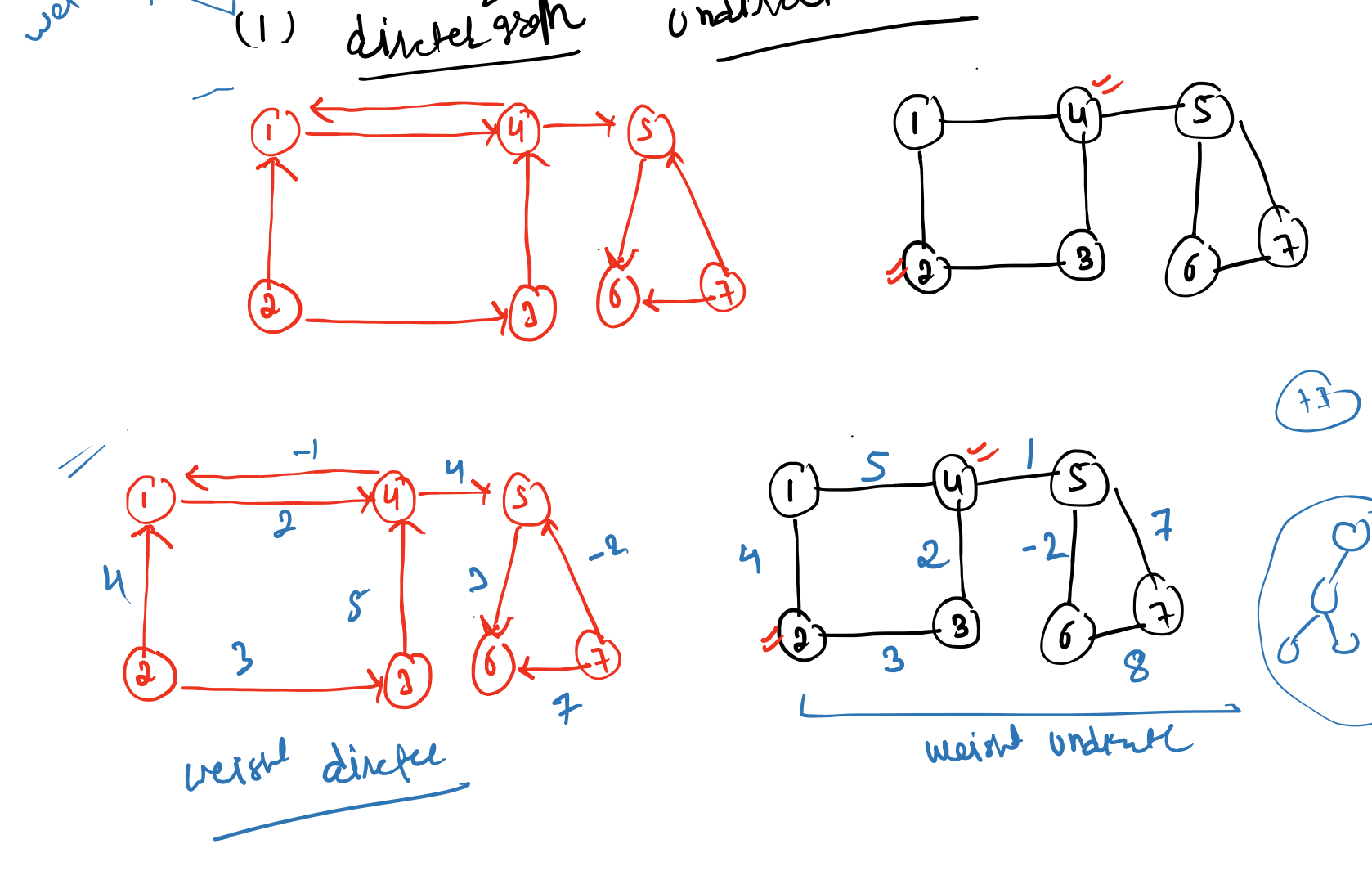

Node - Vtx
Edge
Child

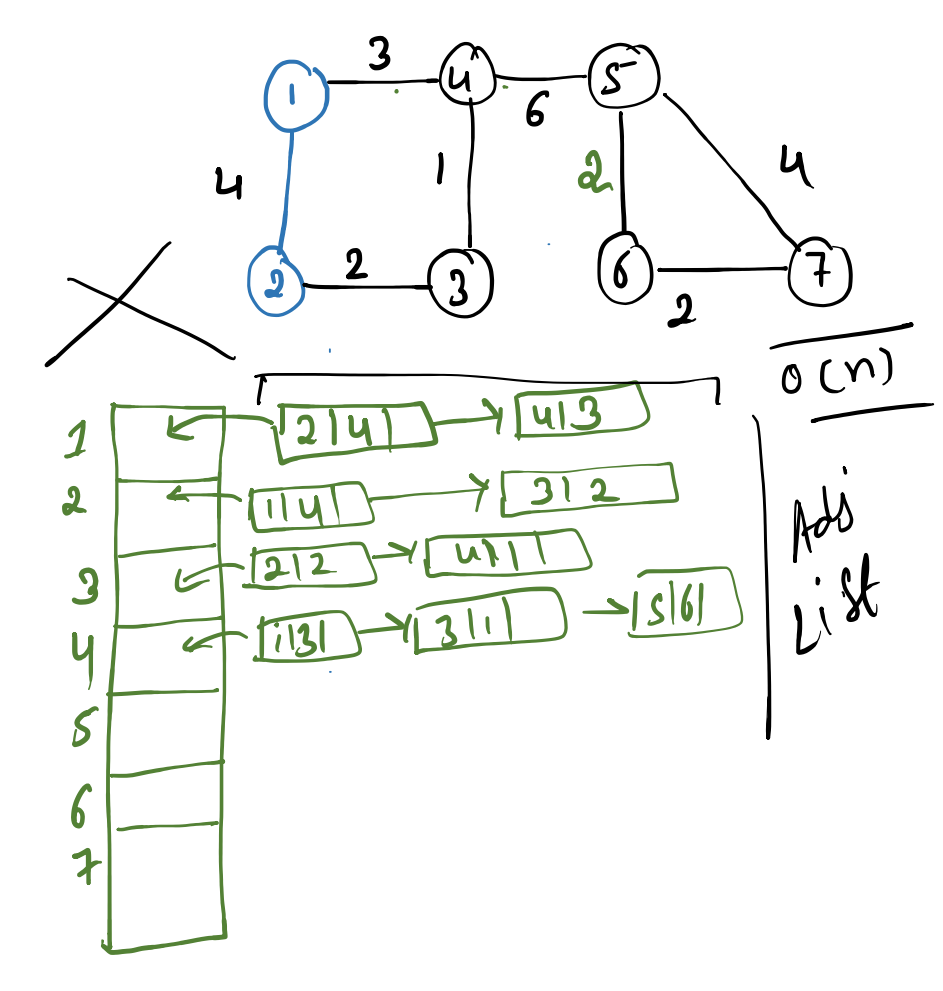
True → Graph ✓
Graph → True ✗

Application ① Routing table ② google map



Graph → (i) directed graph → weight / unweight
Graph → undirected graph → weight / unweight


1
4
5
2
3
6
7
-1
2
4
3
5
-2
7
weight directed
weight undirected
5
1
-2
8

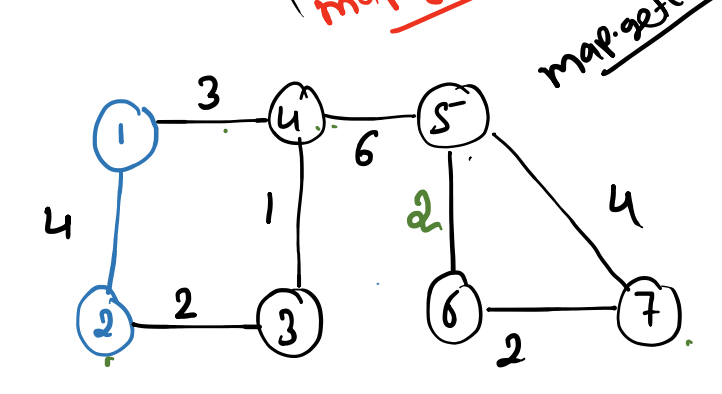

3
6
4
1
2
2
4
2
1
2
3
4
5
6
7
2|4
4|3
1|4
3|2
2|2
4|1
1|3
3|1
5|6
O(n)
Adj List

① Adj matrix ✗

	1	2	3	4	5	6	7
1	0	4	0	3	0	0	0
2	4	0	2	0	0	0	0
3	0	2	0	1	0	0	0
4	3	0	1	0	6	0	0
5	0	0	0	6	0	2	4
6	0	0	0	0	2	0	2
7	0	0	0	0	4	2	0

map.get(v1).remove(v2)
map.get(v1).put(v2, cost)
map.get(v1).containsKey(v2)
nbrs

For(int V: map.keySet()){
sum += map.get(V).size()

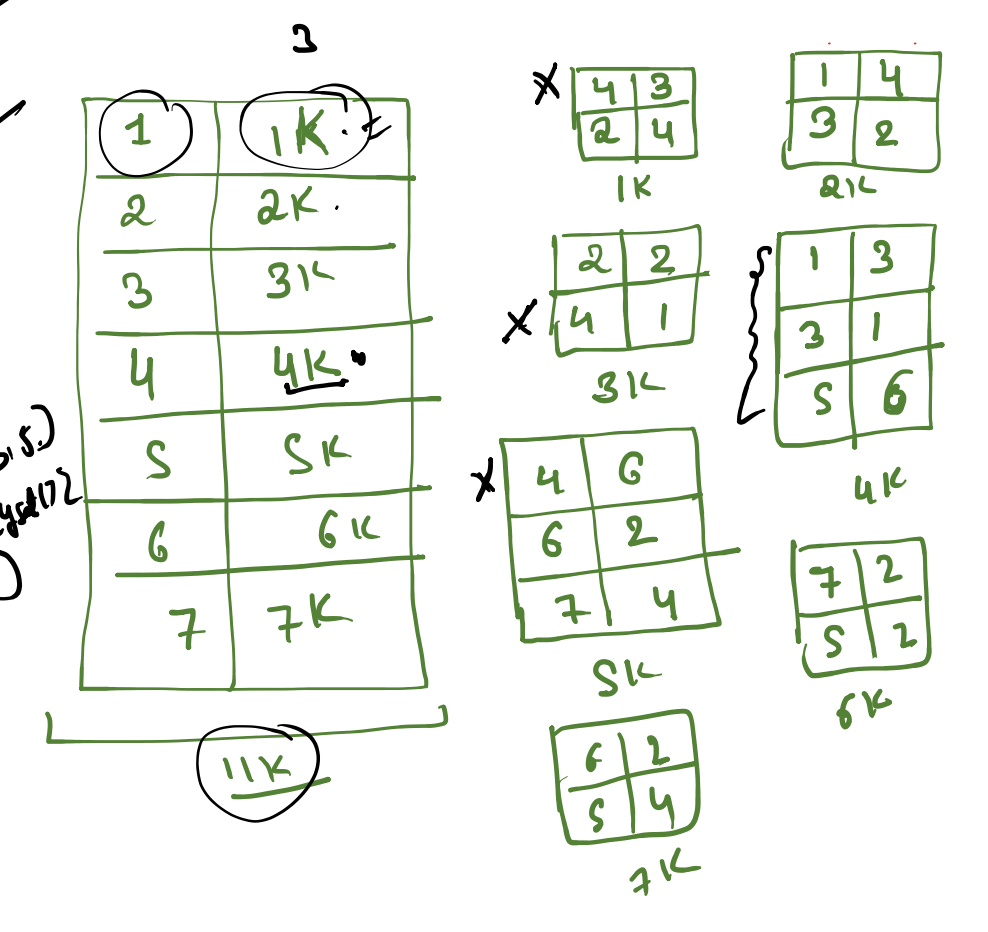


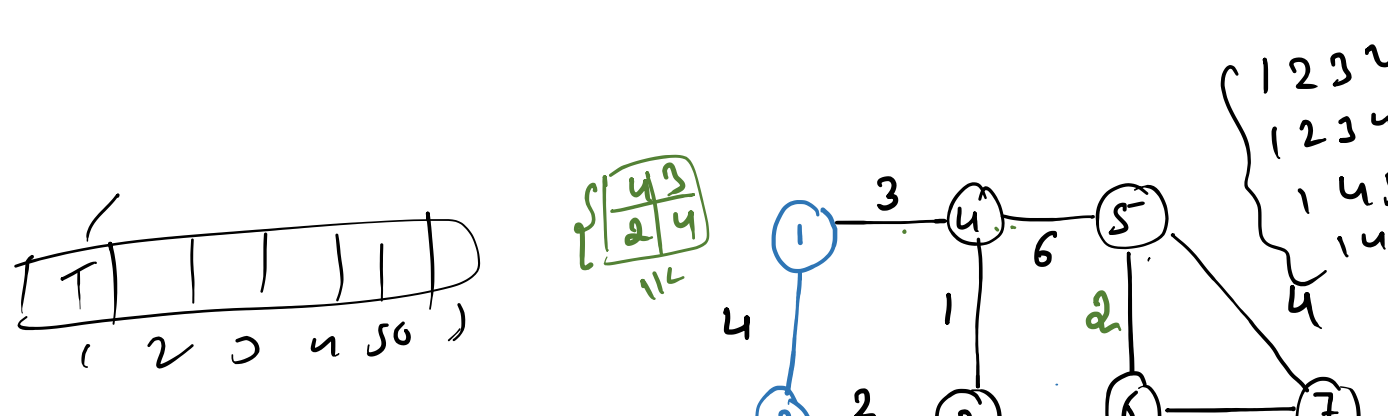
$\frac{2+2+2+3+3+2+2}{2} = \frac{16}{2} = 8$

For(int nbrs: map.get(v).keySet())
map.get(nbrs).remove(v)

1	1K
2	2K
3	3K
4	4K
5	5K
6	6K
7	7K

11K


4 3
2 4
1K
1 4
3 2
2K
2 2
4 1
3K
1 3
3 1
5 6
4K
4 6
6 2
7 4
5K
7 2
5 2
6K
6 2
5 4
7K


T
1 2 3 4 5 6
4 3
2 4
1K
3
6
4
1
2
2
4
2

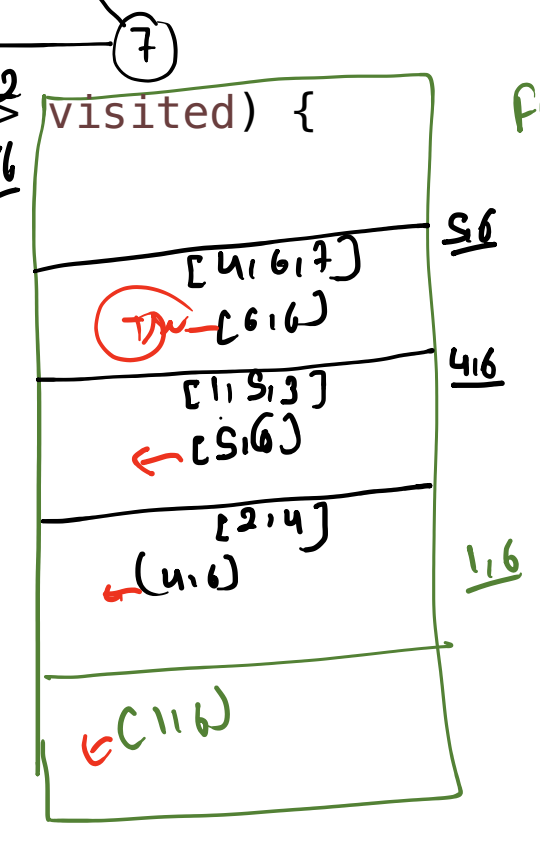
123456
1234576
1456
14576

src == des

```
public boolean HasPath(int src, int des, HashSet<Integer> visited) {
    if (src == des) {
        return true;
    }
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            boolean ans = HasPath(nbrs, des, visited);
            if (ans) {
                return ans;
            }
        }
    }
    return false;
}
```

145

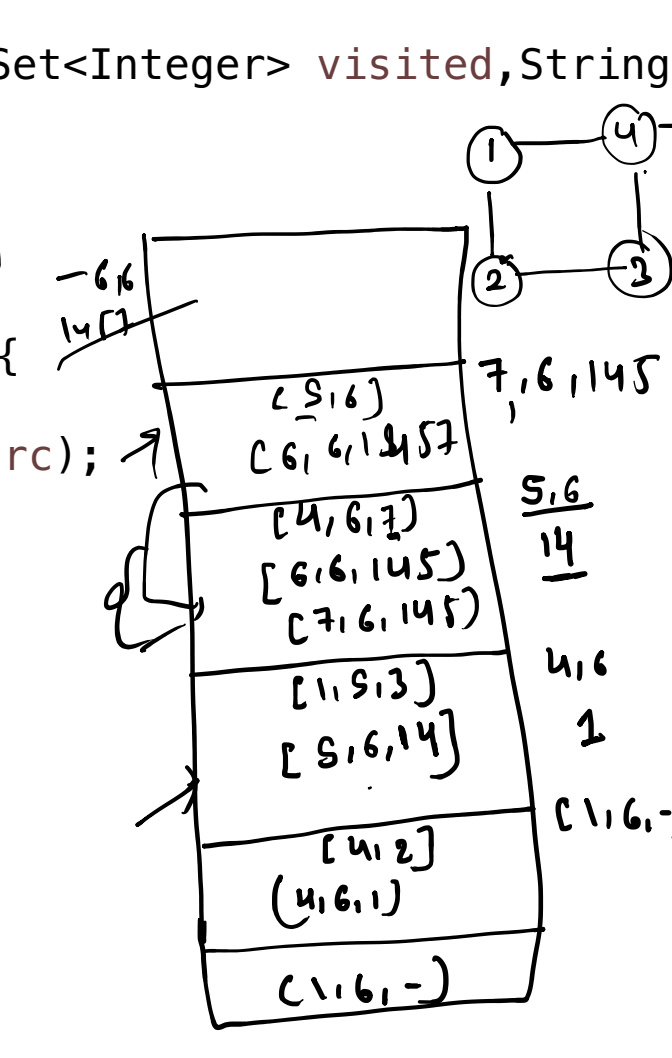
For(int nbrs: map.get(src).keySet()
boolean ans = has(nbrs, des)
ans

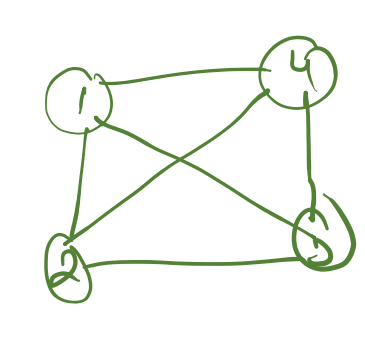

[4,6,7]
True [6,6]
[1,5,3]
[5,6]
[2,4]
(4,6)
(1,6)
5,6
4,6
1,6

```
public void printpath(int src, int des, HashSet<Integer> visited,String ans) {
    if(src == des) {
        System.out.println(ans+des);
        return;
    }
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            printpath(nbrs, des, visited,ans+src);
        }
    }
}
```

145
7

1456
14576


[5,6]
[6,6,1457]
[4,6,1]
[6,6,145]
[7,6,145]
[1,5,3]
[5,6,14]
[4,2]
(4,6,1)
(1,6,-)
7,6,145
5,6
14
4,6
1
[1,6,-]

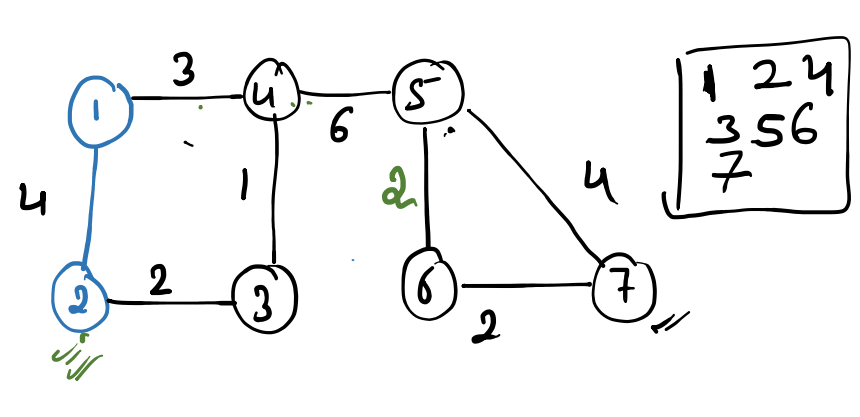


$\frac{(n-1)\times N}{2} = \frac{N\times(N-1)}{2}$

nC_2

$\frac{n!}{(n-2)!\,2!}$

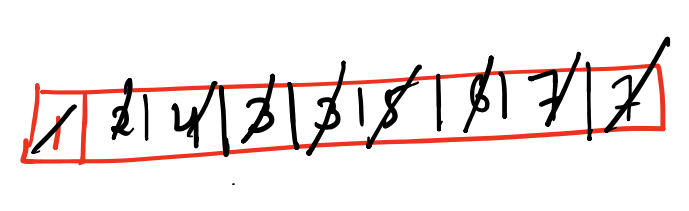
$\frac{n\times(n-1)(n-2)!}{(n-2)!\,2}$

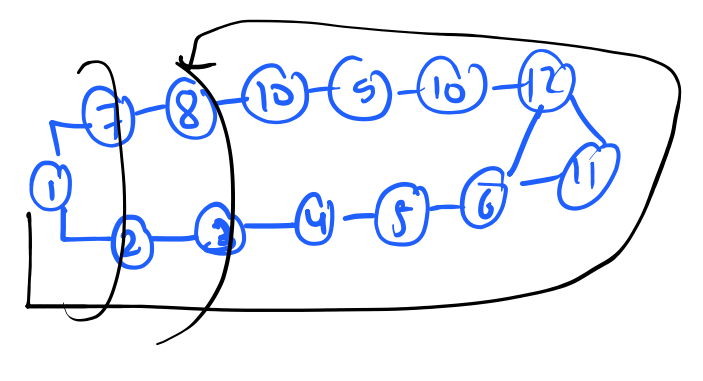


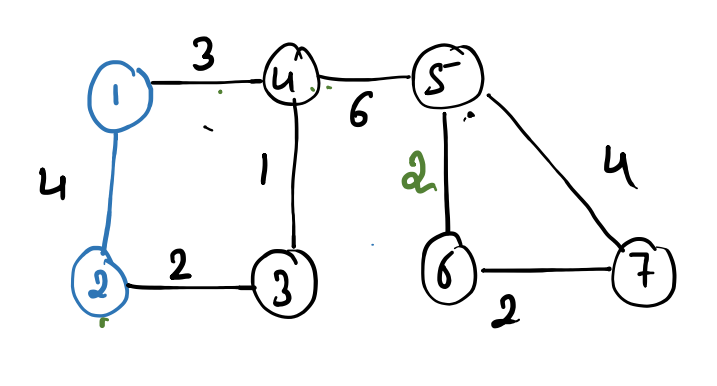
124
356
7

B.F.S | D.F.S

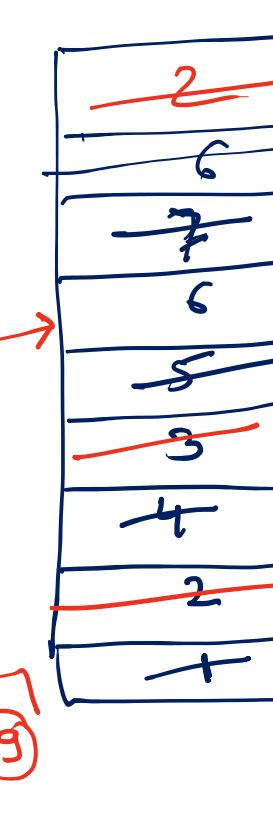
1. remove
2. Ignore if Already visited → cycle
3. marked visited
4. Selfwork
5. Add unvisited nbrs





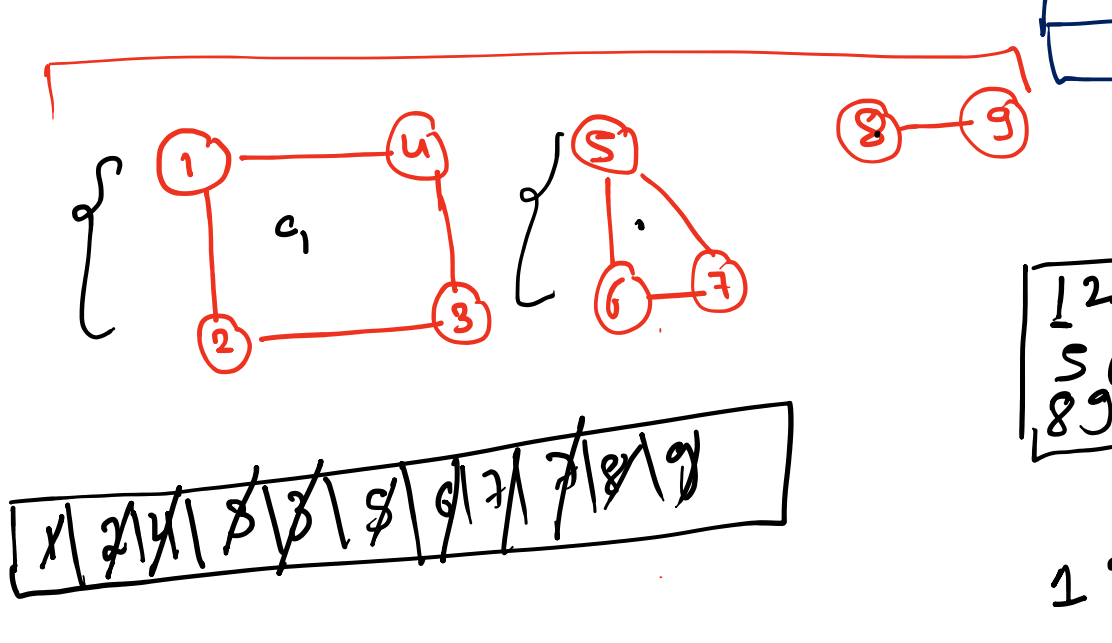


1457
632



1. remove
2. Ignore → cont
3. marked visited
→4. self work
5. Add nbrs

V + E
n
n^2


1
4
2
3
5
6
7
8
9
1 2 4 3 3 5 6 7 7 8 9

1243
567
89

1 2 4 3 5 6 7 8 9

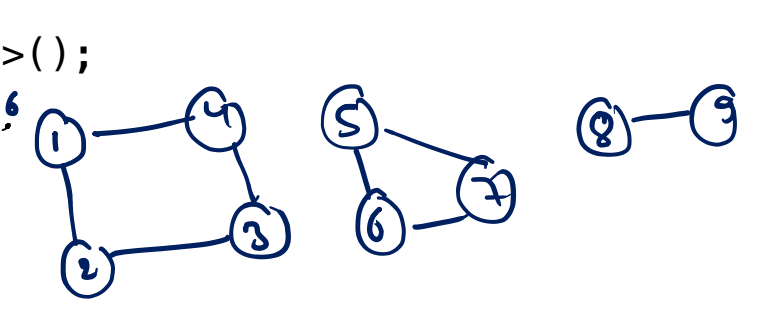
D.F.T → B.F.T
D.F.T

1. remove
2. Ignore
3. marked visited
4. self work
5. Add nbrs

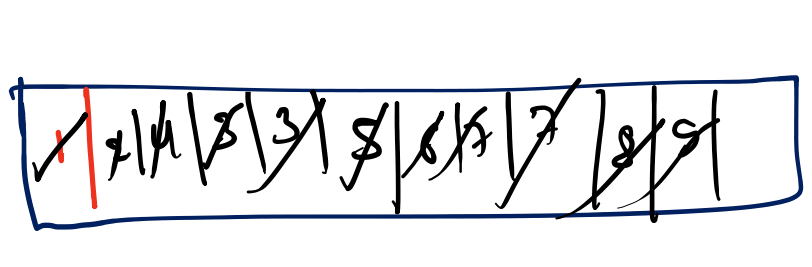
```
public void BFT() {
    Queue<Integer> q = new LinkedList<>();
    HashSet<Integer> visited = new HashSet<>();
    for (int src : map.keySet()) {
        if(visited.contains(src)) {
            continue;
        }
        q.add(src);
        while (!q.isEmpty()) {
            // 1. remove
            int r = q.poll();
            // 2. Ignore if Already visited
            if (visited.contains(r)) {
                continue;
            }
            // 3. Marked visited
            visited.add(r);
            // 4. self work
            System.out.print(r + " ");
            // 5. add unvisited nbrs
            for (int nbrs : map.get(r).keySet()) {
                if (!visited.contains(nbrs)) {
                    q.add(nbrs);
                }
            }
        }
    }
    System.out.println();
}
```

[1,2,3,4,5,6,7,8,9]

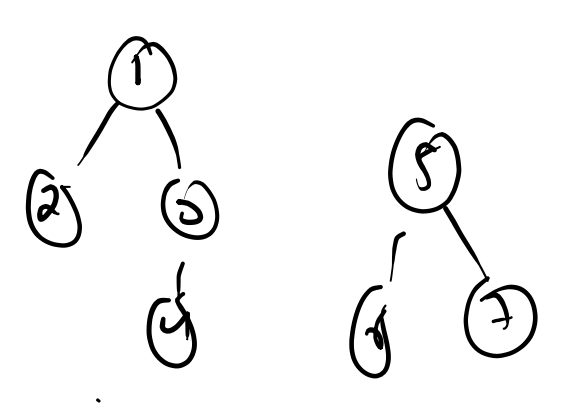
124
3 5
6 7 8
9



1 2 4 3 5 6 7 8 9



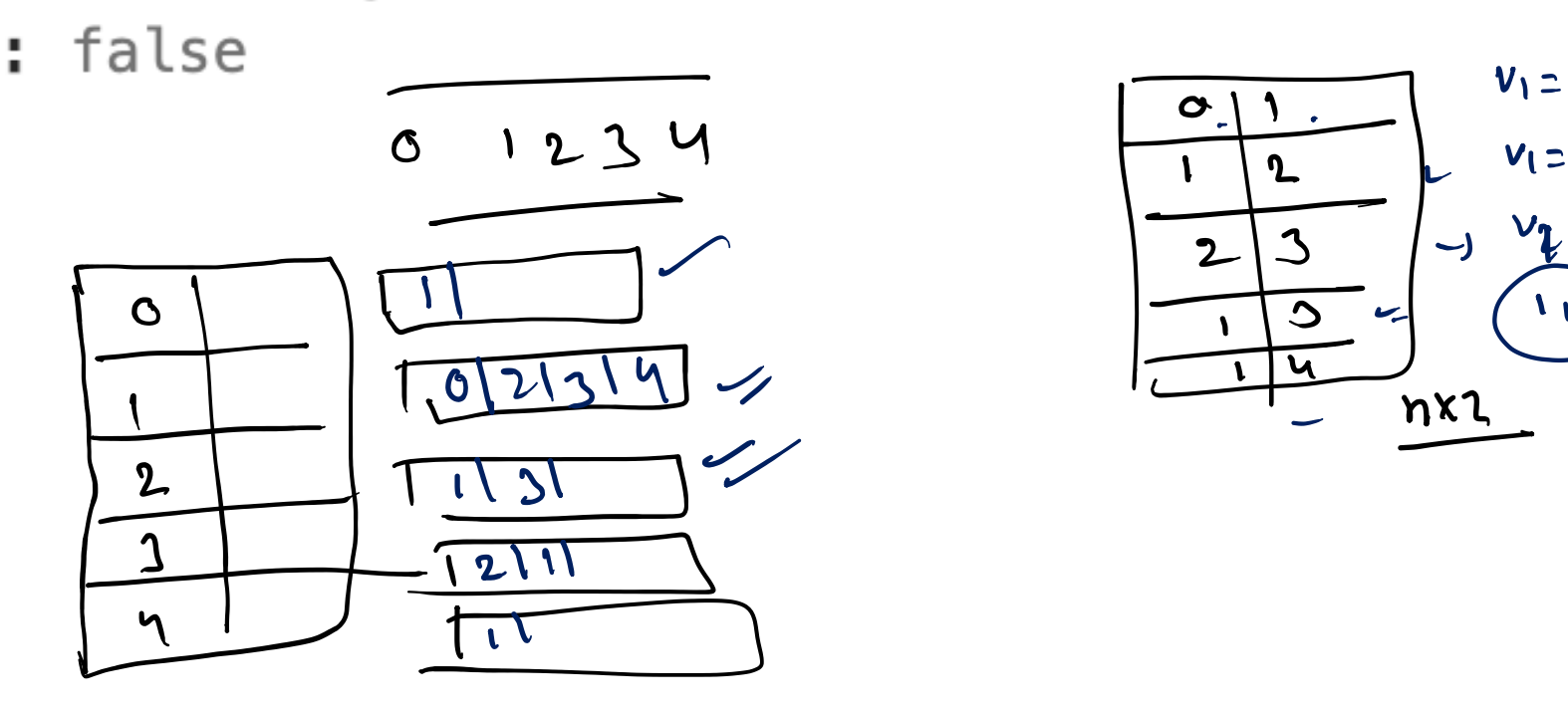
① must



map.get(0).keySet()

```
Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]
Output: false
```

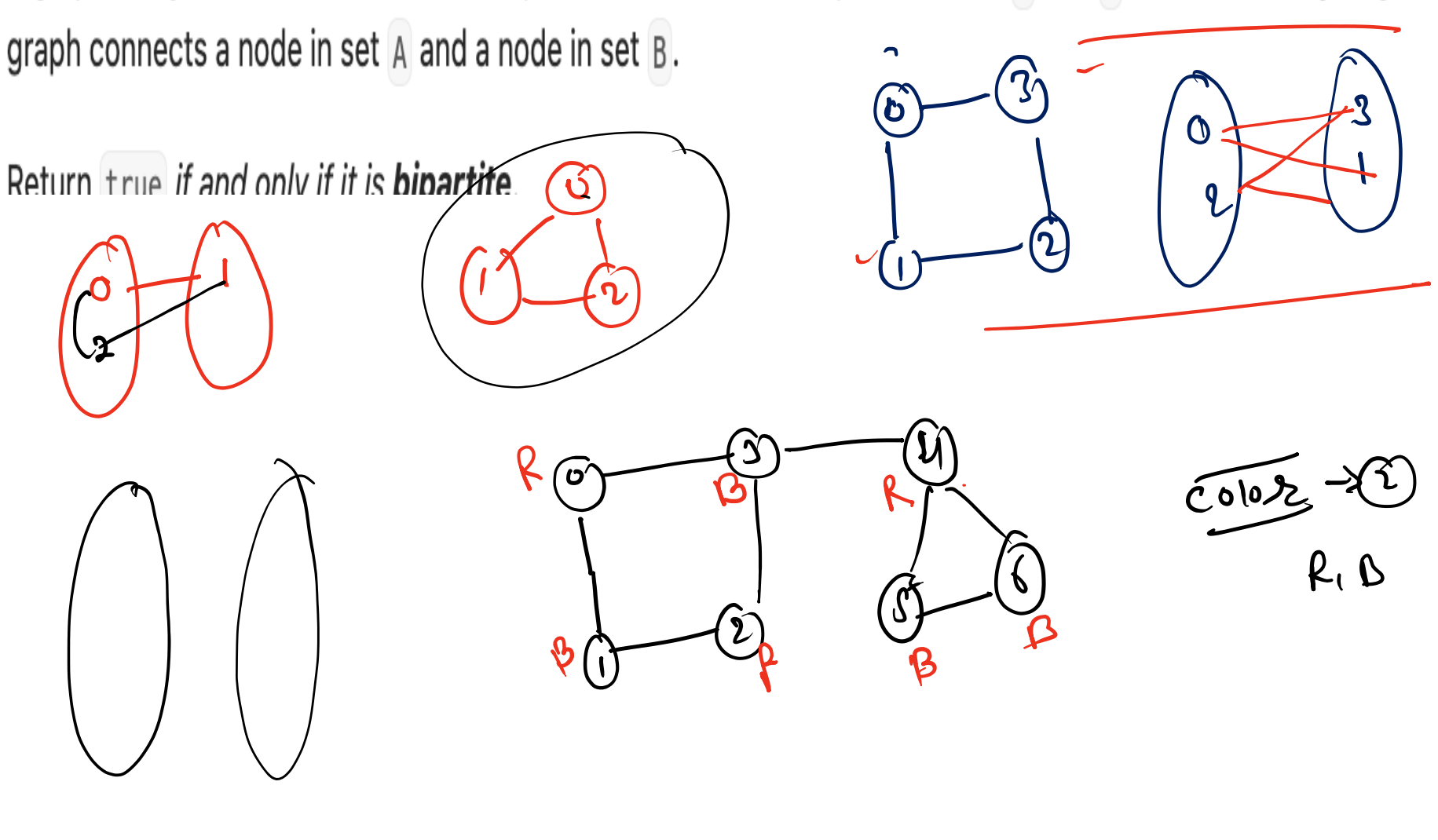
0 1 2 3 4


0
1
2
3
4
1
0 2 3 4
1 3
2 1
1
0 1
1 2
2 3
1 3
1 4
nx2

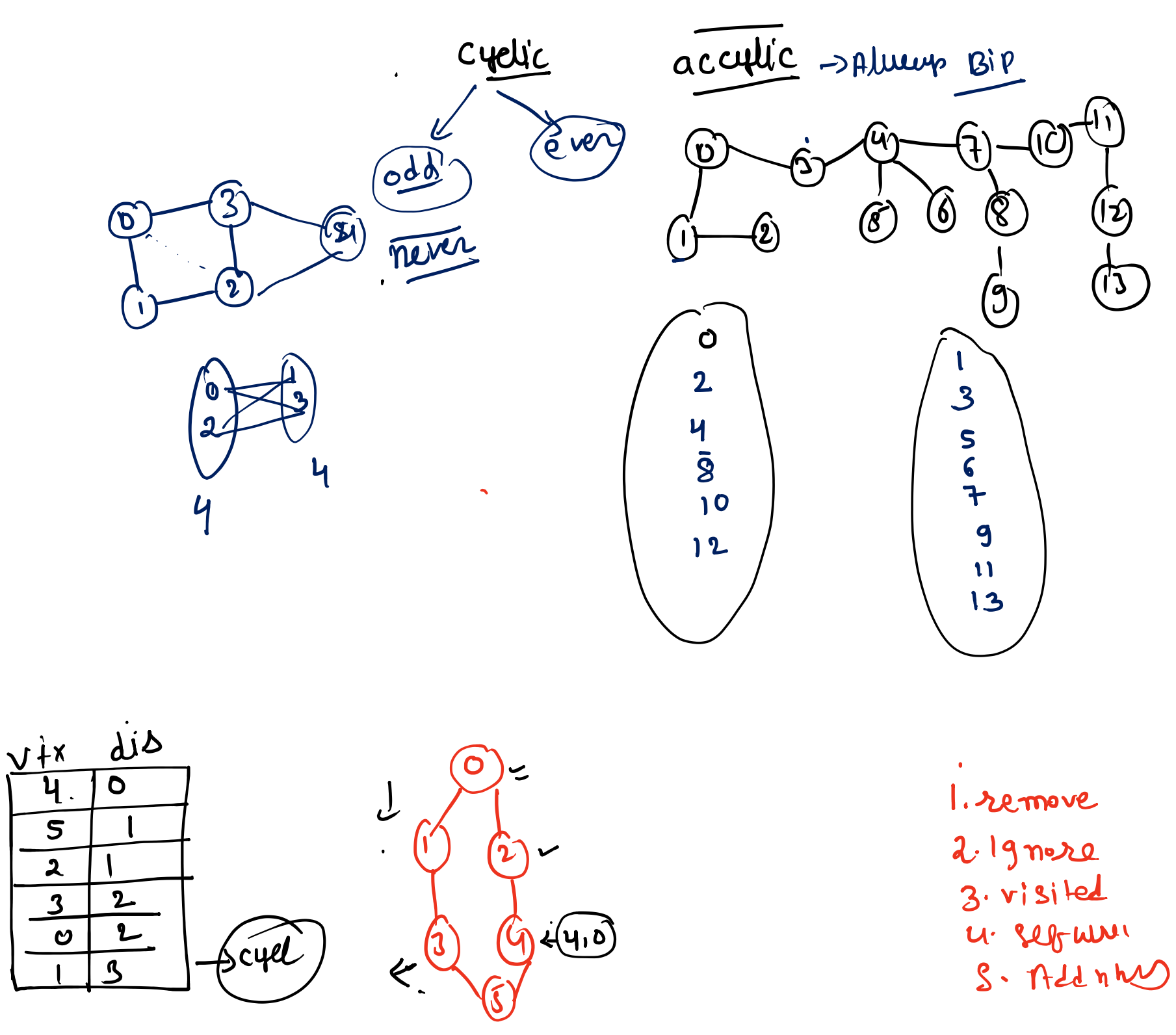
$v_1 = 0 \quad v_1 = 1$
$v_1 = 1 \quad v_2 = 2$
$v_2 = 2 \quad v_2 = 3$

A graph is **bipartite** if the nodes can be partitioned into two independent sets A and B such that **every** edge in the graph connects a node in set A and a node in set B.

Return true if and only if it is bipartite.

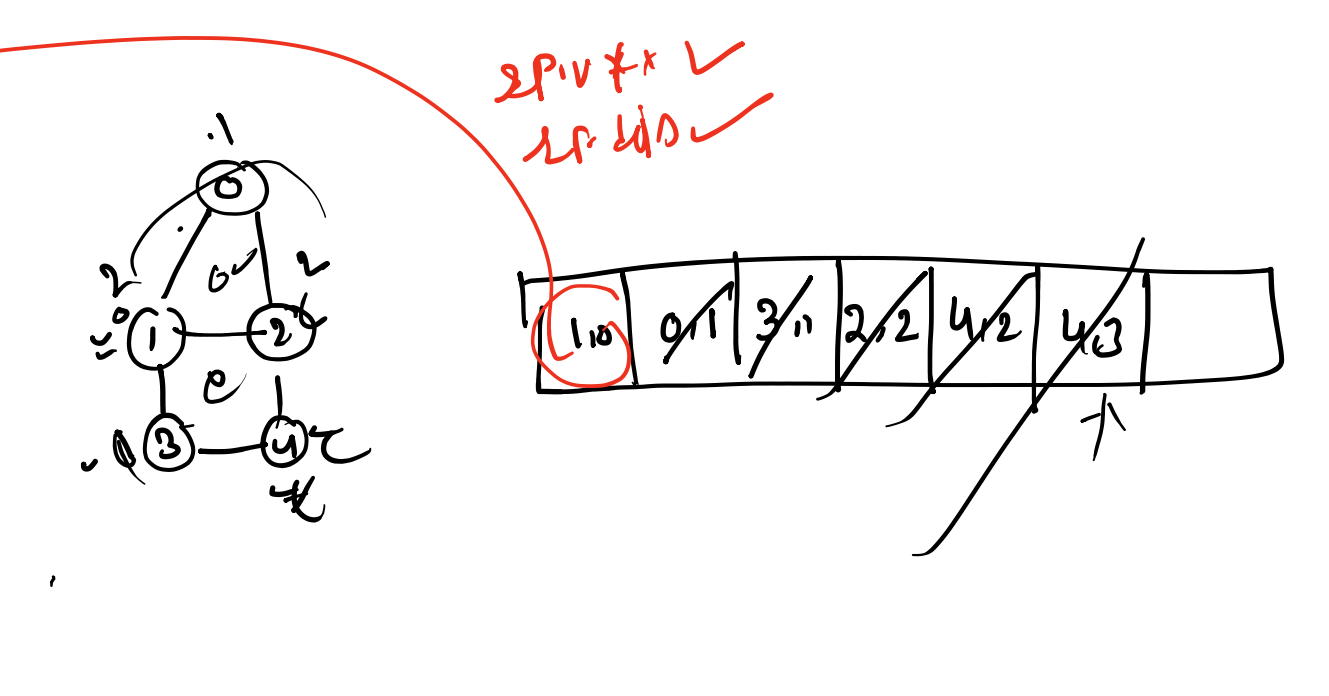

0
1
2
3
0
3
1
2
0
2
3
1
R
B
R
B
R
B
B
R

color → 2
R, B


cyclic
odd
even
never
acyclic → Always Bip
0
2
4
8
10
12
1
3
5
6
7
9
11
13

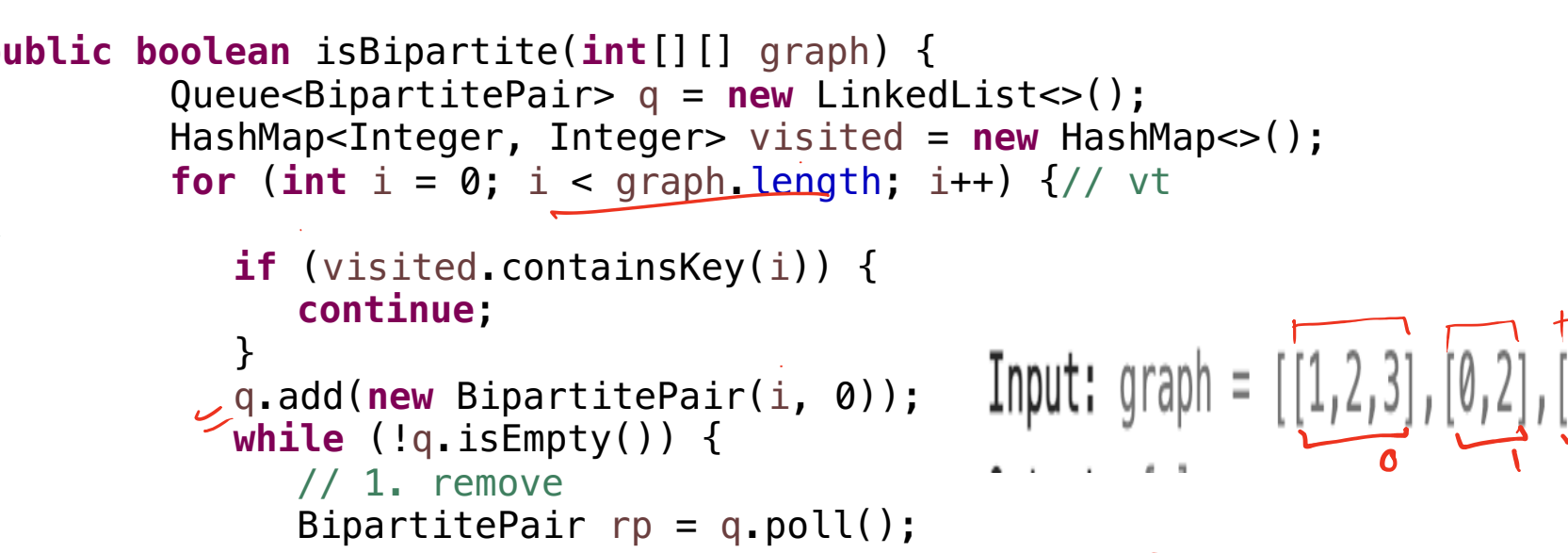
vtx	dis
4	0
5	1
2	1
3	2
0	2
1	3

cycle


0
1
2
3
4
5
(4,0)
(4,0) 5,1 2,1 3,2 0,2 1,3 1,3

1. remove
2. Ignore
3. visited
4. self work
5. Add nbrs

2 pi vtx
1 pi dis

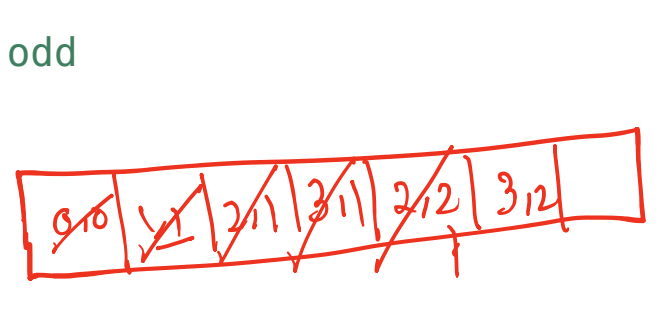

1 0
0 1
3 1
2 2
4 2
0
1
2
3
4
1,0 0,1 3,1 2,2 4,2 4,3

```
public boolean isBipartite(int[][] graph) {
        Queue<BipartitePair> q = new LinkedList<>();
        HashMap<Integer, Integer> visited = new HashMap<>();
        for (int i = 0; i < graph.length; i++) {// vt

            if (visited.containsKey(i)) {
                continue;
            }
            q.add(new BipartitePair(i, 0));
            while (!q.isEmpty()) {
                // 1. remove
                BipartitePair rp = q.poll();
                // 2. Ignore if Already visited
                if (visited.containsKey(rp.vtx)) {
                    if (visited.get(rp.vtx) != rp.dis) {// odd
                        return false;
                    }
                    continue;
                }
                // 3. marked visited
                visited.put(rp.vtx, rp.dis);
                // 4. self work
                // 5. Add unvisited nbrs
                for (int nbrs : graph[rp.vtx]) {
                    if (!visited.containsKey(nbrs)) {
                        q.add(new BipartitePair(nbrs, rp.dis + 1));
                    }
                }
            }
        }
        return true;
    }
```

Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]
0 1 2 3

v	d
0	0
1	1
2	1
3	1


0,0 1,1 2,1 3,1 2,2 3,2